

AL2002 Artificial Intelligence Lab Manual
ver 2.0

Saad Ahmad, BS

Inst., Dept. of Computer Science

National University of Computer & Emerging Sciences

Artificial Intelligence Lab Manual

Saad Ahmad

Instructor, Dept. of Computer Science

National University of Computer & Emerging Sciences

160 Industrial Estate, Peshawar, KPK, Pakistan

Version History

1.0 Spring 2026: Compilation of all previous labs in book format with minor updates & corrections {**Saad Ahmad**}

2.0 Spring 2026: Restructured manual: updated chapter titles, added Simulated Annealing, new CSP chapter (Lab 9), Linear & Logistic Regression in Ch. 13, improved heuristics explanation, added TikZ figures & code snippets {**Saad Ahmad**}

This manual is customized for course AL2002 Artificial Intelligence taught at the National University of Computer & Emerging Sciences, Peshawar, Pakistan. The lab portion of this course provides cover for 1 credit hour. Evaluation for this lab is at discretion of your instructor and/or teacher.

The manual can also be used as a learning tool for other artificial intelligence courses both at undergraduate or graduate level. Each chapter in this manual is intended to serve as a separate lab, although instructors should note that some chapter's require more than one lab sessions to complete.

Contents

I	MACHINE LEARNING	5
1	Supervised Learning: Naive Bayes	7
1.1	What is Naive Bayes?	7
1.2	Understanding Bayes' Theorem	7
1.3	Example 1: Play Tennis Dataset	8
1.3.1	Step 1: Checking the Big Picture (Prior Probability)	8
1.3.2	Step 2: Finding the Clues (Likelihoods)	9
1.3.3	Step 3: Making the Final Guess (Comparing Scores)	9
1.3.3.1	Score for Yes:	9
1.3.3.2	Score for No:	9
1.4	Python Implementation: Step-by-Step	9
1.4.1	Part 1: The Dataset	10
1.4.2	Part 2: Finding Prior Probabilities	10
1.4.3	Part 3: Counting the Features (Likelihoods)	10
1.4.4	Part 4: The Prediction Rule	11
1.5	Example 2: Text Classification (Spam Filter)	12
1.6	Common Pitfalls (Troubleshooting)	12
1.7	Key Concepts Summary	13
1.8	Final Lab Tasks & Challenges	13
1.9	Conclusion	13

Part I

MACHINE LEARNING

Chapter 1

Supervised Learning: Naive Bayes

1.1 What is Naive Bayes?

Naive Bayes is a simple and very popular machine learning algorithm. It is used to classify data. For example, it can decide if an email is "Spam" or "Not Spam", or if the weather is good to "Play Tennis" or "Not Play Tennis".

It is called "**Naive**" because it assumes that all features (input values) do not affect each other. This is a very simple assumption, but it works surprisingly well in many real-world problems.

It is called "**Bayes**" because it uses a math rule called **Bayes' Theorem**, named after Thomas Bayes. This rule helps us find the probability (chance) of something happening, based on what we already know.

1.2 Understanding Bayes' Theorem

Bayes' Theorem helps us update our beliefs when we see new evidence.

Imagine this:

- You know that 1% of people have a rare disease.
- A test for this disease is 99% accurate.
- If a person tests positive, what is the chance they actually have the disease?

Because the disease is so rare, even a 99% accurate test will give many false warnings. Bayes' Theorem helps us calculate the exact math.

The formula for Bayes' Theorem in classification is:

$$P(\text{Class} \mid \text{Features}) = \frac{P(\text{Features} \mid \text{Class}) \times P(\text{Class})}{P(\text{Features})}$$

In simple words:

1. **$P(\text{Class})$ (Prior Probability)**: This is what we believe **before** we look at any new evidence. It is just the general chance of something happening based on past history. *Example: Before looking outside, you know that it usually rains 20% of the time in your city.*
2. **$P(\text{Features} \mid \text{Class})$ (Likelihood)**: This is the chance of seeing these specific features **if** we assume the class is already true. *Example: If we assume it is raining, what is the chance that it is also Cloudy? If clouds almost always come with rain, this number will be very high.*

3. $P(\text{Class} \mid \text{Features})$ (**Posterior Probability**): This is our **final guess**. It is the updated chance of the class being true after we have combined our past knowledge (Prior) with the new evidence (Likelihood).

To find the winner, we calculate a score for every possible class (like "Yes" and "No"). Since we are only comparing these scores to see which one is bigger, we can ignore the bottom part of the formula ($P(\text{Features})$) because it is the same for everyone. This makes our math much faster!

We calculate our final **Winning Score** by multiplying our chain of chances together:

- We start with the **Prior Probability** (what we already knew).
- We multiply it by the **Likelihood** of the first feature.
- We multiply that by the **Likelihood** of the second feature, and so on for every piece of evidence we have.

The scoring formula looks like this:

$$\text{Final Score} = P(\text{Class}) \times P(\text{Feature 1} \mid \text{Class}) \times P(\text{Feature 2} \mid \text{Class}) \dots$$

1.3 Example 1: Play Tennis Dataset

Let us look at a famous small dataset about playing tennis. We want to predict if we should **Play Tennis (Yes or No)** based on the weather.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Table 1.1: Play Tennis Dataset

1.3.1 Step 1: Checking the Big Picture (Prior Probability)

First, we look at our past 14 days and count how often we played tennis. This gives us our "baseline" probability for any random day.

In our dataset, there are 9 "Yes" days and 5 "No" days.

- **General chance of Yes:** $9/14 = 0.64$
- **General chance of No:** $5/14 = 0.36$

1.3.2 Step 2: Finding the Clues (Likelihoods)

Now, we look at the specific weather features. We want to know: "If we decided to play (or not play), what was the weather like?" We calculate this for every feature like Outlook, Temperature, and Wind.

For example, let's look at the **Outlook** clue:

- On the 9 days we played tennis (**Yes**), how many were Sunny? **2 days**. So, the chance of Sunny when we play is $2/9 = 0.22$.
- On the 5 days we did **not** play (**No**), how many were Sunny? **3 days**. So, the chance of Sunny when we don't play is $3/5 = 0.60$.

We do this exact same counting for Temperature, Humidity, and Wind to get all our clues ready.

1.3.3 Step 3: Making the Final Guess (Comparing Scores)

Now, imagine a new day with this weather: **Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong**. Should we play?

We calculate a score for both "Yes" and "No" by multiplying our clues together.

1.3.3.1 Score for Yes:

We multiply the base chance (Step 1) by all the clues for "Yes" weather (Step 2):

$$\begin{aligned}\text{Score(Yes)} &= P(\text{Yes}) \times P(\text{Sunny} \mid \text{Yes}) \times P(\text{Cool} \mid \text{Yes}) \times P(\text{High} \mid \text{Yes}) \times P(\text{Strong} \mid \text{Yes}) \\ &= 0.64 \times 0.22 \times 0.33 \times 0.33 \times 0.33 \\ &\approx \mathbf{0.0053}\end{aligned}$$

1.3.3.2 Score for No:

Now we do the same for "No":

$$\begin{aligned}\text{Score(No)} &= P(\text{No}) \times P(\text{Sunny} \mid \text{No}) \times P(\text{Cool} \mid \text{No}) \times P(\text{High} \mid \text{No}) \times P(\text{Strong} \mid \text{No}) \\ &= 0.36 \times 0.60 \times 0.20 \times 0.80 \times 0.60 \\ &\approx \mathbf{0.0207}\end{aligned}$$

The Result: Since the score for **No** (0.0207) is higher than the score for **Yes** (0.0053), our classification for this new day is **No** (Do not play tennis).

1.4 Python Implementation: Step-by-Step

As a student, the best way to understand an algorithm is to build it yourself. In this section, we will write Naive Bayes using only basic Python lists and dictionaries. We will not use Pandas or Scikit-Learn. We will build it block by block.

1.4.1 Part 1: The Dataset

```

1 # We store our dataset as a list of lists.
2 # Columns: [Outlook, Temperature, Humidity, Wind, PlayTennis]
3 data = [
4     ["Sunny", "Hot", "High", "Weak", "No"],
5     ["Sunny", "Hot", "High", "Strong", "No"],
6     ["Overcast", "Hot", "High", "Weak", "Yes"],
7     ["Rain", "Mild", "High", "Weak", "Yes"],
8     ["Rain", "Cool", "Normal", "Weak", "Yes"],
9     ["Rain", "Cool", "Normal", "Strong", "No"],
10    ["Overcast", "Cool", "Normal", "Strong", "Yes"],
11    ["Sunny", "Mild", "High", "Weak", "No"],
12    ["Sunny", "Cool", "Normal", "Weak", "Yes"],
13    ["Rain", "Mild", "Normal", "Weak", "Yes"],
14    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
15    ["Overcast", "Mild", "High", "Strong", "Yes"],
16    ["Overcast", "Hot", "Normal", "Weak", "Yes"],
17    ["Rain", "Mild", "High", "Strong", "No"]
18 ]

```

What is happening here? We are literally copying our weather table into Python code. Every row in our data is a simple list of features. The very last item in each list (like “Yes” or “No”) is the correct answer that we want our computer to learn.

Student Activity 1: Before moving forward, write a small Python for loop that goes through the data variable and only prints out the **Outlook** of each day.

1.4.2 Part 2: Finding Prior Probabilities

Next, we need to know the basic overall chances. Out of 14 days, how many are “Yes” and how many are “No”?

```

1 total_samples = len(data)
2 class_counts = {"Yes": 0, "No": 0}
3
4 for row in data:
5     final_answer = row[-1]
6     class_counts[final_answer] += 1
7
8 prior_yes = class_counts["Yes"] / total_samples
9 prior_no = class_counts["No"] / total_samples
10
11 print(f"Total Yes Days: {class_counts['Yes']} -> Prior Yes: {prior_yes:.3f}")
12 print(f"Total No Days: {class_counts['No']} -> Prior No: {prior_no:.3f}")

```

What is happening here? Inside this code, we create a counter (a dictionary) starting at zero. We loop through every single day in our data. If a day ends with “Yes”, we add 1 to our Yes counter. Finally, we divide our counts by the total number of days (14). This gives us our **baseline chance** (0.64 for Yes and 0.36 for No) which is the starting point for our brain.

1.4.3 Part 3: Counting the Features (Likelihoods)

Now we must find out how many times “Sunny” happened on a “Yes” day, and how many times “Hot” happened on a “No” day, and so on.

```

1 feature_counts_yes = {}
2 feature_counts_no = {}
3
4 for row in data:
5     final_answer = row[-1]
6
7     for i in range(len(row) - 1):
8         feature = row[i]
9
10        if final_answer == "Yes":
11            if feature not in feature_counts_yes:
12                feature_counts_yes[feature] = 0
13                feature_counts_yes[feature] += 1
14
15        elif final_answer == "No":
16            if feature not in feature_counts_no:
17                feature_counts_no[feature] = 0
18                feature_counts_no[feature] += 1
19
20 print("Features on Yes days:", feature_counts_yes)
21 print("Features on No days:", feature_counts_no)

```

What is happening here? This is the "counting" part. We use two different dictionaries to keep track of words seen on Yes days vs No days. Our loop scans every row. If it's a Yes day, it looks at every word (like "Sunny" or "Hot") and adds it to the list of "Yes clues". For example, `feature_counts_yes["Sunny"]` will equal 2 because "Sunny" appeared exactly twice on days we played tennis.

Student Activity 2: Run the code above. Look at the printed dictionary for "Yes" days. How many times did the word "High" appear on a "Yes" day? Verify if the code is correct by manually counting it from the table in Section 3.

1.4.4 Part 4: The Prediction Rule

We have all the numbers. Let's build a function to predict an entirely new test day!

```

1 def predict_naive_bayes(new_day):
2     score_yes = prior_yes
3     score_no = prior_no
4
5     for feature in new_day:
6         # Multiply Yes score
7         if feature in feature_counts_yes:
8             prob = feature_counts_yes[feature] / class_counts["Yes"]
9             score_yes = score_yes * prob
10        else:
11            score_yes = score_yes * 0.01
12
13        # Multiply No score
14        if feature in feature_counts_no:
15            prob = feature_counts_no[feature] / class_counts["No"]
16            score_no = score_no * prob
17        else:
18            score_no = score_no * 0.01
19
20    print(f"Final Score for Yes: {score_yes:.5f}")

```

```

21     print(f"Final Score for No: {score_no:.5f}")
22
23     if score_yes > score_no:
24         print("Prediction: Yes, play tennis!")
25     else:
26         print("Prediction: No, do not play tennis.")

```

What is happening here? To predict a new day, we pass a list of features (like “Sunny”, “Cool”) into our function. The computer starts with our baseline chance (0.64). Then, it looks up every “clue” from our dictionaries and **multiplies** the score. It does this once for Yes and once for No. The side that ends up with the biggest final score is our winner!

1.5 Example 2: Text Classification (Spam Filter)

One of the most famous uses of Naive Bayes is **detecting Spam emails**.

When finding spam, we don’t look at weather. Instead, our features are the **words** inside the email! If an email contains words like “Free”, “Money”, or “Winner”, the chance of it being Spam goes way up.

```

1 emails = [
2     ["win", "free", "money", "now", "SPAM"],
3     ["hello", "mom", "call", "me", "NOT SPAM"],
4     ["free", "gift", "click", "here", "SPAM"],
5     ["meeting", "at", "ten", "tomorrow", "NOT SPAM"],
6     ["win", "winner", "prize", "money", "SPAM"]
7 ]

```

What is happening here? Instead of weather, we use **words** as our features. The logic is identical: we count how many times “Win” or “Free” appears in Spam emails vs. Normal emails. We then multiply those chances to see if a new message has the “vibe” of a spam message or not.

Student Activity 3: Imagine you receive a new email that just says: ["free", "money", "meeting"]. Based on your own intuition, do you think it is Spam or Not Spam? Use the logic of the code we learned to create a prediction function for these text messages!

1.6 Common Pitfalls (Troubleshooting)

- **Capital Letters Matter:** Python thinks “Sunny” and “sunny” are different things. For best results, it is a good idea to convert all your text to **lowercase** before counting!
- **The Zero Problem:** If a word appears in the training data for “Yes” but never for “No”, the score for No will become zero. This isn’t fair! Professional developers use **Laplace Smoothing** (adding +1 to all counts) to make sure every outcome has at least a tiny chance.
- **Missing Columns:** Always make sure your test data has the exact same categories as your training data, or your code will get confused and crash.

1.7 Key Concepts Summary

- **Prior Probability:** Your starting "guess" based on general history (the big picture).
- **Likelihood:** The clues we get from specific features (the details).
- **Posterior Probability:** Your final decision after combining the big picture with the details.
- **Laplace Smoothing:** A simple trick to avoid multiplying by zero by adding +1 to every count.

1.8 Final Lab Tasks & Challenges

1. **Experiment 1: The Paper Test.** Grab a calculator! Pick a row from the data table and try to find the winner using just your hands and brain. Does your math match the computer's math?
2. **Experiment 2: Messing with the Data.** Add three new rows to the `data` list. Make them very weird (like playing tennis when it's raining). See how this changes the scores when you test a new day.
3. **Experiment 3: Testing Every Day.** Write a loop that tests all 14 days in our table. Does our simple algorithm get them all right? Calculate the **Accuracy** (Correct Guesses / Total Days).
4. **Challenge: The Real CSV Task.** Create a file called `student_data.csv` in your folder. Put some fake data inside (like StudyHours, Attendance, Result). For this task, you are allowed to use the `pandas` library to load your data.

Tip: You can load your CSV and convert it to a list of lists using:

```
1 import pandas as pd
2 df = pd.read_csv("student_data.csv")
3 data = df.values.tolist()
```

Now, use your custom Naive Bayes code to predict if a student will pass or fail based on this new data!

1.9 Conclusion

Naive Bayes is a "fast and simple" champion in the world of Machine Learning. While it is "naive" because it makes simple assumptions, it remains one of the most efficient tools for things like text classification and fraud detection. By building it from scratch today, you have learned the true engine inside this algorithm!

Bibliography

- [1] Agents in artificial intelligence.
- [2] Artificial intelligence.
- [3] Artificial intelligence.
- [4] Artificial intelligence - agents and environments.
- [5] Cs6-1 lab 06.
- [6] Networkx - python software package for complex networks.
- [7] Types of artificial intelligence.
- [8] What is intelligent agent in ai - types and function.